Core 90 - Graph traversal: BFS and DFS

8 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

7	1
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Core 90 - Graph traversal: BFS and DFS | 8 questions | tap a title to jump to it

M	1. Path Minimizing the Maximum Cell (Swim in Rising Water)	1Point3Acres	2026-09-29
M	2. Rotten Oranges / Multi-Source BFS (taxis)	1Point3Acres	2026-09-08
M	3. Count Islands in a Grid and Track the Count as Land Cells Are Added	PracHub	2026-10-02
M	4. Count Clusters of 2D Points Within a Radius	PracHub	2026-06-18
M	5. Solve Grid and Counting Problems	PracHub	2025-01-25
M	6. Find shortest relationship path using BFS	PracHub	2026-02-12
M	7. Validate course catalog dependencies	PracHub	2025-08-01
H	8. Infection Spread on a Grid (Cellular Automaton)	PracHub	2026-06-14

STUDY PATH: question 1 of 167 (Core 90)

Priority score 7.6 | Graph traversal: BFS and DFS | asked 21 days ago | frequency Low

MEDIUM | IN APP LC 778 | 1Point3Acres | onsite-coding | 1 of 8

Path Minimizing the Maximum Cell (Swim in Rising Water)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 55/228

ABOUT THIS QUESTION

Topic: Core 90 - Graph traversal: BFS and DFS

Optimal answer from: your app's interview approach (LeetCode #778)

LeetCode match: LC 778 Swim in Rising Water

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

QUESTION

A bottleneck-path family: minimize the maximum value encountered on a grid path, then generalize the same reasoning to a weighted graph whose edges carry safety values and an admissibility threshold.

Grid variant

Input is a grid of heights and start/end cells; movement is through four-neighbors.
Cost of a path is the maximum cell value along it.
Return the minimum possible path cost.

Weighted-graph safety variant

Input is a graph whose edges carry safety values, a start node, an end node, and a safety threshold.
Find the lowest safety value required by any valid start-to-end path while every traversed edge remains under the supplied threshold.
Return -1 when no admissible path exists.
Clarify whether the threshold comparison is strict or inclusive before coding.

Mouse-and-cat safest-path variant (L4 phone screen)

Warm-up: an N x N grid with a start S, a target T, and impassable water cells; with 4-directional moves, decide whether any path from S to T exists.
Main question: the grid also contains a cat at C. The mouse moves only on land, 4-directionally, from S to the cheese at T. Find a path that maximizes the minimum Manhattan distance to the cat over all cells on the path.
This flips the objective (maximize a minimum instead of minimize a maximum), but it is the same bottleneck-path structure once each cell's distance to the cat is known.

Follow-up: Network Delay Time variant

In a directed graph with edge weights, return the maximum shortest-path distance from a source to all nodes, or -1 if any node is unreachable.

QUESTION

EXAMPLES

Grid

```
[ 0  1  2  3  4 ]
[24 23 22 21  5 ]
[12 13 14 15 16 ]
[11 17 18 19 20 ]
[10  9  8  7  6 ]
```

Start=(0,0), End=(4,0) in (row, col) — the bottom-left cell with value 10. Answer: 16 (the path's max cell is 16).

APPROACH

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, not the sum. That cost can only grow as the path grows, so Dijkstra still works if you replace 'distance + weight' by max(distance, weight): the first time a cell leaves the heap its cost is final. The cat version flips the objective (maximise a minimum), which is the same shape with a max-heap and each cell scored by its distance to the cat.

Basis: LeetCode 778, Swim in Rising Water (bottleneck path), plus the 'safest path from a cat' flip: maximise the minimum Manhattan distance.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, start, end):
    for t in sorted(set(all_values(grid))):
        if bfs_reaches(grid, start, end, t):
            return t  # smallest limit
```

TIME	SPACE
O(V^2)	O(V)

WHY IT IS SLOW

One full BFS per distinct value, O(V^2). Dijkstra on the running maximum finds the answer in one pass, O(V log V).

OPTIMAL SOLUTION · Dijkstra on the running maximum

TIME	SPACE
O(V log V)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Maximise the minimum distance to a cat instead.

Same bottleneck shape with the objective flipped: score each cell by its Manhattan distance to the cat, run Dijkstra with a max-heap on the running minimum, and skip water. The same monotonicity argument holds (extending a path can only lower its minimum).

FOLLOW-UP 2 Why is Dijkstra correct for a max-along-path cost?

The cost max(path) never decreases when a path is extended, so a vertex's cheapest cost cannot be improved by going through a vertex that is popped later. That is the only property Dijkstra needs from its cost function (monotone, non-negative extension).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Path Minimizing the Maximum Cell (Swim in
# Rising Water)
#
# A bottleneck-path family: minimize the maximum value
# encountered on a grid path, then generalize the same
# reasoning to a weighted graph whose edges carry safety
# values and an admissibility threshold. Grid variant
# Input is a grid of heights and start/end cells; movement
# is through four-neighbors. Cost of a path is the maximum
# cell value along it. Return the minimum possible path
# cost. Weighted-graph safety variant Input is a graph
# whose edges carry safety values, a start node, an end
# node, and a safety threshold. Find the lowest safety
# value required by any valid start-to-end path while
# every traversed edge remains under the supplied
# threshold. Return -1 when no admissible path exists.
# Clarify whether the threshold comparison is strict or
# inclusive before coding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Maximise the minimum distance to a cat instead.
# 2. Why is Dijkstra correct for a max-along-path cost?
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, not the sum.
# That cost can only grow as the path grows, so Dijkstra
# still works if you replace 'distance + weight' by
# max(distance, weight): the first time a cell leaves the
# heap its cost is final. The cat version flips the
# objective (maximise a minimum), which is the same shape
# with a max-heap and each cell scored by its distance to
# the cat.
#
# BRUTE FORCE (say it first): Try every threshold and test
# reachability
# For each candidate value t in increasing order run a BFS
# using only cells <= t and stop at the first t that
# connects start to end. Correct, but it repeats a full
# search per value. Time O(V^2). Space O(V).


from collections import deque
import heapq
```

```
# BRUTE FORCE (say it first): Try every threshold and test
# reachability | time O(V^2)
def bottleneck_brute(g, start, end):
    n, m = len(g), len(g[0])
    for t in sorted({x for row in g for x in row}):
        if (
            g[start[0]][start[1]] > t
            or g[end[0]][end[1]] > t
        ):
            continue
        seen, q = {start}, deque([start])
        while q:
            r, c = q.popleft()
            if (r, c) == end:
                return t
            for a, b in (
                (r + 1, c),
                (r - 1, c),
                (r, c + 1),
                (r, c - 1),
            ):
                if (
                    0 <= a < n
                    and 0 <= b < m
                    and (a, b) not in seen
                    and g[a][b] <= t
                ):
                    seen.add((a, b))
                    q.append((a, b))
    return None


# HELPERS for the base question
INF = float("inf")


def around(r, c, n, m):
    # four-neighbour cells inside an n x m grid
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < n and 0 <= b < m:
            yield a, b


# BASE QUESTION (optimal): Dijkstra on the running maximum
```

```
# Idea: Heap entries are (cost so far, cell). Popping a
# cell fixes its cost; each neighbour gets max(cost,
# neighbour value). Stop when the end cell is popped. Skip
# stale heap entries.
def min_max_path(g, start, end):
    n, m = len(g), len(g[0])
    # best[cell] = smallest possible "highest cell so far"
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, (r, c) = heapq.heappop(heap)
        # skip outdated heap entries
        if cost > best[(r, c)]:
            continue
        if (r, c) == end:
            return cost
        for a, b in around(r, c, n, m):
            # path cost = highest cell so far or this cell
            nxt = max(cost, g[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, (a, b)))
    return None
```

```python
# VARIANT: weighted graph with a safety threshold
def min_bottleneck_edge(graph, s, t, limit):
    # graph[u] = [(v, safety)]; every edge must be < limit
    best = {s: 0}
    heap = [(0, s)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:
            continue
        if u == t:
            return cost
        for v, w in graph.get(u, []):
            # edges at or above the limit are not allowed
            if w >= limit:
                continue
            nxt = max(cost, w)
            if nxt < best.get(v, INF):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    # no admissible path
    return -1


# RELATED: network delay (plain Dijkstra)
def network_delay(n, edges, src):
    # plain Dijkstra: largest shortest distance, or -1
    graph = {}
    for u, v, w in edges:
        graph.setdefault(u, []).append((v, w))
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    # unreachable nodes make the answer -1
    return max(dist.values()) if len(dist) == n else -1


# FOLLOW-UP 1: Maximise the minimum distance to a cat
# instead.
# Answer: Same bottleneck shape with the objective
# flipped: score each cell by its Manhattan distance to
# the cat, run Dijkstra with a max-heap on the running
# minimum, and skip water.
def safest_path_score(grid, start, target, cat):
    # maximise the smallest cat distance ('#' is water)
    n, m = len(grid), len(grid[0])

    def dist(r, c):
        return abs(r - cat[0]) + abs(c - cat[1])

    best = {start: dist(*start)}
    heap = [(-best[start], start)]
    while heap:
        neg, (r, c) = heapq.heappop(heap)
        if -neg < best[(r, c)]:
            continue
        if (r, c) == target:
            return -neg
        for a, b in around(r, c, n, m):
            if grid[a][b] == "#":
                continue
            # the path is only as safe as its closest cell
            score = min(-neg, dist(a, b))
            if score > best.get((a, b), -1):
                best[(a, b)] = score
                heapq.heappush(heap, (-score, (a, b)))
    return None


if __name__ == "__main__":
    g = [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5],
         [12, 13, 14, 15, 16], [11, 17, 18, 19, 20],
         [10, 9, 8, 7, 6]]
    assert min_max_path(g, (0, 0), (4, 0)) == 16
    # start equals end: just that cell
    assert min_max_path(g, (2, 2), (2, 2)) == 14
    gr = {0: [(1, 5), (2, 9)], 1: [(2, 3)]}
    assert min_bottleneck_edge(gr, 0, 2, 10) == 5
    assert min_bottleneck_edge(gr, 0, 2, 5) == -1
    cat_grid = ["...", ".#.", "..."]
    cat, a, b = (0, 2), (0, 0), (2, 2)
    score = safest_path_score(cat_grid, a, b, cat)
    assert score == 2
    chain = [(0, 1, 2), (1, 2, 3)]
    assert network_delay(3, chain, 0) == 5
    assert network_delay(3, [(0, 1, 2)], 0) == -1
```

NOTES FROM CANDIDATES

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter.

Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

In the mouse-and-cat variant, each cell's score is a direct Manhattan distance to a single cat, so it can be computed per cell; clarify whether water cells block only the mouse or also affect distance, and whether S or T themselves count toward the minimum.

PREPARATION

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

STUDY PATH: question 2 of 167 (Core 90)

Priority score 7.05 | Graph traversal: BFS and DFS | asked 42 days ago | frequency Low

MEDIUM | IN APP LC 994 | 1Point3Acres | onsite-coding, phone-screen | 2 of 8

Rotten Oranges / Multi-Source BFS (taxis)

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 0 | PRIORITY 67/228

ABOUT THIS QUESTION

Topic: Core 90 - Graph traversal: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Also in your app: LeetCode #994 Rotting Oranges

LeetCode match: LC 994 Rotting Oranges

Frequency: Low | Last asked: 2026-09-08 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/rotten-oranges-multi-source-bfs-taxis

QUESTION

EXAMPLES

```
Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges
    variant): return number of minutes until all
    reachable 1s become 2 (4).
```

Taxi variant: for each empty cell, distance to nearest taxi.

APPROACH

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

HOW TO THINK ABOUT IT

'Spread outward from several starting points at the same time and count the rounds' is multi-source BFS: put every source in the queue before you start, and process the queue one round at a time. Rounds are minutes. Always count the fresh ones first so that unreachable oranges are detected at the end.

Basis: LeetCode 994, Rotting Oranges, and the same multi-source BFS skeleton as LeetCode 542 and 286.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    minutes = 0
    while True:
        fresh = fresh_next_to_rotten(grid)
        if not fresh:
            break
        for r, c in fresh:
            grid[r][c] = 2
        minutes += 1
    return -1 if any_fresh(grid) else minutes
```

TIME	SPACE
O(minutes * m * n)	O(m * n)

WHY IT IS SLOW

Each minute scans the entire grid, O(minutes * m * n). A queue of the newly rotten oranges touches each cell only once, O(m * n).

OPTIMAL SOLUTION · Multi-source BFS by minute

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Taxi variant: distance from every empty cell to the nearest taxi

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

FOLLOW-UP 2 Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

FOLLOW-UP 3 Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

FOLLOW-UP 4 A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Rotten Oranges / Multi-Source BFS (taxis)
#
# Examples: Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-
# oranges variant): return number of minutes until all
# reachable 1s become 2 (4). Taxi variant: for each empty
# cell, distance to nearest taxi.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Sources move or new taxis appear. How do you keep the
# distances fresh?
# 2. Why is multi-source BFS the same cost as single-
# source BFS?
# 3. Signal variant: a signal spreads from sources, how
# long until every target gets it?
# 4. A signal spreads from sources to targets: how long
# until every target is reached?
#
# INTERVIEW NOTES
# 'Spread outward from several starting points at the same
# time and count the rounds' is multi-source BFS: put
# every source in the queue before you start, and process
# the queue one round at a time. Rounds are minutes.
# Always count the fresh ones first so that unreachable
# oranges are detected at the end.
#
# BRUTE FORCE (say it first): Re-scan the whole grid every
# minute
# Each minute, find every fresh orange next to a rotten
# one and rot them all at once, until nothing changes.
# Easy to reason about and the oracle; each minute touches
# the entire grid. Time O(minutes * m * n). Space O(m *
# n).


from collections import deque


# BRUTE FORCE (say it first): Re-scan the whole grid every
# minute | time O(minutes * m * n)
def oranges_by_scanning(grid):
    g = [row[:] for row in grid]
    minutes = 0
    while True:
        rot = [
            (r, c)
            for r in range(len(g))
            for c in range(len(g[0]))
            if g[r][c] == 1
            and any(
                0 <= r + dr < len(g)
                and 0 <= c + dc < len(g[0])
                and g[r + dr][c + dc] == 2
                for dr, dc in (
                    (1, 0),
                    (-1, 0),
                    (0, 1),
                    (0, -1),
                )
            )
        ]
        if not rot:
            break
        for r, c in rot:
            g[r][c] = 2
        minutes += 1
    return -1 if any(1 in row for row in g) else minutes


# HELPERS for the base question
def multi_source_bfs(rows, cols, sources, open_cell):
    # distance from the nearest source to every cell
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c),
                       (r, c + 1), (r, c - 1)):
            inside = 0 <= nr < rows and 0 <= nc < cols
            if inside and (nr, nc) not in dist:
                if open_cell(nr, nc):
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
    return dist


# BASE QUESTION (optimal): Multi-source BFS by minute
# Idea: Push every rotten orange into the queue at minute
# 0 and expand level by level; each level is one minute.
# Count the fresh oranges and decrement as they rot, so
# the answer is -1 if any remain at the end.
def rotting_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    cells = [(r, c) for r in range(rows)
             for c in range(cols)]
    rotten = [x for x in cells if grid[x[0]][x[1]] == 2]
    fresh = [x for x in cells if grid[x[0]][x[1]] == 1]
    # rot spreads through any non-empty cell
    dist = multi_source_bfs(
        rows, cols, rotten, lambda r, c: grid[r][c] != 0)
    # a fresh orange that is never reached means -1
    if any(x not in dist for x in fresh):
        return -1
    return max((dist[x] for x in fresh), default=0)


# TAXI VARIANT: distance to the nearest taxi
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked
    rows, cols = len(grid), len(grid[0])
    taxis = [(r, c) for r in range(rows)
             for c in range(cols) if grid[r][c] == "T"]
    dist = multi_source_bfs(
        rows, cols, taxis, lambda r, c: grid[r][c] != "#")
    # None where no taxi can reach the cell
    return [[dist.get((r, c)) for c in range(cols)]
```

```
            for r in range(rows)]


# FOLLOW-UP 4: A signal spreads from sources to targets:
# how long until every target is reached?
# Answer: One multi-source BFS from the sources, then the
# answer is the largest distance over the targets (-1 if
# any target is unreachable, 0 if there are no targets).
# Per-source BFS would cost O(mn * sources).
def signal_time(rows, cols, sources, targets, blocked=()):
    # how long until every target hears the signal?
    dist = multi_source_bfs(
        rows, cols, list(sources),
        lambda r, c: (r, c) not in blocked)
    if any(t not in dist for t in targets):
        return -1
    # no targets means no waiting
    return max((dist[t] for t in targets), default=0)
```

```
if __name__ == "__main__":
    assert rotting_minutes([[2, 1, 1], [1, 1, 0],
                            [0, 1, 1]]) == 4
    # a fresh orange walled off by an empty cell
    assert rotting_minutes([[2, 0, 1]]) == -1
    assert rotting_minutes([[0, 2]]) == 0
    assert nearest_taxi(["T.#", "..."]) == [
        [0, 1, None], [1, 2, 3]]
    # signal from two sources to two targets
    assert signal_time(1, 5, [(0, 0), (0, 4)],
                       [(0, 1), (0, 2)]) == 2
    assert signal_time(1, 3, [(0, 0)], [(0, 2)],
                       blocked={(0, 1)}) == -1
    assert signal_time(1, 3, [(0, 0)], []) == 0
```

NOTES FROM CANDIDATES

BFS from all sources simultaneously is O(mn); per-source BFS is O(mn × #sources).
The moving-source follow-up trades exactness for amortized cost — common answer is periodic rebuild with bounded staleness.
For the signal variant, use the maximum source-distance over targets as the completion time; an unreachable target makes that candidate infeasible. Agree that no targets requires zero time. A correct baseline for destroying at most one blocker is to run multi-source BFS with no deletion, then once with each blocker made traversable, restoring it before the next trial. Minimize the completion time across feasible trials; return -1 if none is feasible. With B blockers on an R by C grid, this takes O((B + 1) * R * C) time and O(R * C) reusable auxiliary space. A BFS state that allows each path to break its own blocker does not enforce one shared destroyed blocker for all targets.

PREPARATION

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

STUDY PATH: question 3 of 167 (Core 90)

Priority score 6.44 | Graph traversal: BFS and DFS | asked 18 days ago

MEDIUM | IN APP LC 305 | PracHub | Technical Screen | 3 of 8

Count Islands in a Grid and Track the Count as Land Cells Are Added

POSTED 2026-10-02 | 18 DAYS AGO | LAST 30 DAYS

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 104/228

ABOUT THIS QUESTION

Topic: Core 90 - Graph traversal: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #305 Number of Islands II

LeetCode match: LC 305 Number of Islands II

Posted on PracHub: 2026-10-02 (updated 2026-10-03)

Original: https://prachub.com/coding-questions/count-islands-in-a-grid-and-track-the-count-as-land-cells-are-added

QUESTION

A map is a grid with **m** rows and **n** columns in which every cell is land (**1**) or water (**0**). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- **grid[r][c]** is **1** for land and **0** for water, where **r** is the row index and **c** is the column index.
- An addition **(r, c)** turns cell **(r, c)** into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length **len(additions) + 1**. Element **0** is the number of islands in the initial grid. For **i >= 1**, element **i** is the number of islands after the first **i** additions have been applied.

Constraints

- **1 <= m <= 1000**, **1 <= n <= 1000** and **m * n <= 200000**
- Every row of **grid** has exactly **n** entries, each **0** or **1**.
- **0 <= len(additions) <= 200000**
- Every addition **(r, c)** satisfies **0 <= r < m** and **0 <= c < n**. The same cell may appear more than once.
- Every count is at most **m * n**, so all values fit in a 32-bit signed integer.

Examples

QUESTION

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

APPROACH

Union-find over cells: start with initial islands, then each addition creates a component and unions with the four neighbors; track the component count. Ignore repeated cells. O((mn + k) alpha).

HOW TO THINK ABOUT IT

Re-running a flood fill after every addition costs O(mn) each time. Land only ever appears, so islands only merge: Union-Find fits exactly. A new land cell starts as a new island (count + 1) and every union with a neighbouring land cell that belongs to a different island removes one island (count - 1).

Basis: LeetCode 305, Number of Islands II. Report the count before any addition and after each; adding a cell that is already land changes nothing.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_land(r, c):
    grid[r][c] = 1
    return count_islands(grid)  # full flood fill
answers = [add_land(r, c) for r, c in additions]
```

TIME	SPACE
O(A * m * n)	O(m * n)

WHY IT IS SLOW

BRUTE FORCE

Each addition costs a pass over the whole grid, O(A * m * n). Union-Find merges the new cell with its land neighbours and updates the count in near-constant time.

OPTIMAL SOLUTION · Union-Find updated cell by cell

TIME	SPACE
O(m * n + A * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Land can also be removed (or turned back to water).

Union-Find cannot split sets. Either recompute the affected island with a local flood fill, or process all operations offline with a segment tree over time and a rollback Union-Find.

FOLLOW-UP 2 The grid is 10^9 x 10^9 with a few additions.

Keep the Union-Find on a dictionary keyed by coordinates, as the code does; memory depends on the number of land cells only, never on the grid size. **Cost:** O(a * alpha) for a additions and O(a) memory, independent of the grid size.

FOLLOW-UP 3 Report the size of the largest island after each addition.

Store a size per root in the Union-Find (union by size) and keep a running maximum, which only grows when land is added; each step stays near-constant time. Checked against recomputing the largest island from scratch. **Cost:** O(alpha) per addition with size tracking; the maximum is one running variable.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Islands in a Grid and Track the
# Count as Land Cells Are Added
#
# A map is a grid with m rows and n columns in which every
# cell is land (1) or water (0). An island is a maximal
# group of land cells connected horizontally or
# vertically. Cells that touch only at a corner are not
# connected. You are given the initial map and a sequence
# of additions, each of which turns one cell into land.
# Report the number of islands before any addition, and
# again after each addition. ### Function Signature python
# def island_counts(grid: list[list[int]], additions:
# list[tuple[int, int]]) -> list[int]: ### Rules -
# grid[r][c] is 1 for land and 0 for water, where r is the
# row index and c is the column index. - An addition (r,
# c) turns cell (r, c) into land. If that cell is already
# land, either initially or because of an earlier
# addition, the map does not change. - Additions are
# applied in the given order. Land never turns back into
# water.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Land can also be removed (or turned back to water).
# 2. The grid is 10^9 x 10^9 with a few additions.
# 3. Report the size of the largest island after each
# addition.
#
# INTERVIEW NOTES
# Re-running a flood fill after every addition costs O(mn)
# each time. Land only ever appears, so islands only
# merge: Union-Find fits exactly. A new land cell starts
# as a new island (count + 1) and every union with a
# neighbouring land cell that belongs to a different
# island removes one island (count - 1).
#
# BRUTE FORCE (say it first): Recount all islands after
# every addition
# Apply the addition and run a full flood-fill count
# again. Correct and the oracle; every addition costs a
# pass over the entire grid. Time O(A * m * n). Space O(m
# * n).


# BRUTE FORCE (say it first): Recount all islands after
# every addition | time O(A * m * n)
```

```
def _count_islands(grid):
    m, n = len(grid), len(grid[0]) if grid else 0
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in seen:
                count += 1
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    for nx, ny in (
                        (x + 1, y),
                        (x - 1, y),
                        (x, y + 1),
                        (x, y - 1),
                    ):
                        if (
                            0 <= nx < m
                            and 0 <= ny < n
                            and grid[nx][ny] == 1
                            and (nx, ny) not in seen
                        ):
                            seen.add((nx, ny))
                            stack.append((nx, ny))
    return count


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def island_counts_recount(grid, additions):
    g = [row[:] for row in grid]
    out = [_count_islands(g)]
    for r, c in additions:
        g[r][c] = 1
        out.append(_count_islands(g))
    return out


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
```

```
# already land; otherwise make it a new set (count + 1)
```

```
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
class Islands:
    def __init__(self):
        # land cell -> parent cell, and sizes of the roots
        self.parent, self.size = {}, {}
        self.count, self.largest = 0, 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add(self, r, c):
        if (r, c) in self.parent:
            return
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1
        self.largest = max(self.largest, 1)
        # join with every land neighbour
        for dr, dc in STEPS:
            nb = (r + dr, c + dc)
            if nb in self.parent:
                self.union((r, c), nb)

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
        # union by size: the small island joins the big
        # one
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        self.count -= 1
        self.largest = max(self.largest, self.size[a])


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
```

```
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
def island_counts(grid, additions):
    # counts before any addition, then after each one
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.count]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.count)
    return out


# FOLLOW-UP 3: Report the size of the largest island after
# each addition.
# Answer: Store a size per root in the Union-Find (union
# by size) and keep a running maximum, which only grows
# when land is added; each step stays near-constant time.
# Checked against recomputing the largest island from
# scratch.
def largest_after_each(grid, additions):
```

```
    # the follow-up: size of the biggest island each time
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.largest]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.largest)
    return out


if __name__ == "__main__":
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    # 2 islands; the adds join them into one; a repeat is
    # ignored
    assert island_counts(grid, adds) == [2, 2, 2, 1, 1]
    sizes = largest_after_each(grid, adds)
```

```
    assert sizes == [1, 2, 3, 5, 5]
    # corners do not connect cells
    assert island_counts([[1, 0], [0, 1]], []) == [2]
    # the grid may be huge: only land cells are stored
    big = Islands()
    big.add(10 ** 9, 10 ** 9)
    big.add(10 ** 9, 10 ** 9 + 1)
    assert big.count == 1 and big.largest == 2
```

FUNCTION AND TEST CASES

Signature

```
def island_counts(grid: list[list[int]], additions: list[l
ist[int]]) -> list[int]:
```

Sample call

```
island_counts([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]],
[])
```

Test cases

Input	Expected
`([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], [])`	`[3]`
`([[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0], [0, 1], [1, 2], [2, 1], [1, 1], [1, 1]])`	`[0, 1, 1, 2, 3, 1, 1]`

STUDY PATH: question 4 of 167 (Core 90)

Priority score 4.63 | Graph traversal: BFS and DFS | asked 124 days ago

MEDIUM | PracHub | Technical Screen | 4 of 8

Count Clusters of 2D Points Within a Radius

POSTED 2026-06-18 | 124 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 40.7** **30 DAYS 0** **PRIORITY 90/228**

ABOUT THIS QUESTION

Topic: Core 90 - Graph traversal: BFS and DFS

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (inferred from overview)

Posted on PracHub: 2026-06-18 (updated 2026-07-01)

Original: https://prachub.com/coding-questions/count-clusters-of-2d-points-within-a-radius

QUESTION

Overview: This question tests graph traversal skills, specifically identifying connected components among a set of 2D points using distance-based adjacency. It assesses the ability to model a geometric problem as a graph and apply DFS, BFS, or Union-Find, a common evaluation of algorithmic thinking in coding interviews. It also probes awareness of efficiency trade-offs when scaling to large point sets.

APPROACH

Union-find over points with distance <= r; count roots. Use squared distance; spatial grid hashing for speed.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the distance is Euclidean, whether the radius is inclusive (a distance of at most r connects), whether points are integers, and the number of points.

BRUTE FORCE TO SAY OUT LOUD

Union-Find over all pairs of points, comparing every pair. O(n^2) distance checks.

OPTIMAL IDEA

Compare squared distances to r squared (no square roots, exact for integers). For many points hash them into grid cells of side r so each point only checks its own and the eight neighboring cells; use Union-Find to merge and count the components with distinct roots. About O(n) expected for evenly spread points.

DRY-RUN THESE CASES

No points, one point, two points exactly r apart (connected if inclusive), a chain of points each close to the next (one cluster), duplicate points.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Dynamic points, a very large radius, or reporting the largest cluster.

PITFALLS

Using the cell size smaller than r so close points fall in non-adjacent cells. Comparing floating-point distances for equality.

HOW TO THINK ABOUT IT

Model it as a graph where points within r are neighbours and count connected components with Union-Find. The all-pairs check is O(n^2); to speed it up, hash points into square cells of side r so that only points in the same or an adjacent cell can be within r. Compare squared distances to r * r to avoid square roots.

Basis: Connected components of a distance graph (LeetCode 547 idea). Two points are connected when their Euclidean distance is at most r; a cluster is a connected component.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
parent = list(range(n))
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x
for a, b in combinations(range(n), 2):
    if dist2(points[a], points[b]) <= r * r:
        parent[find(a)] = find(b)
clusters = len({find(i) for i in range(n)})
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n^2 / 2 distance checks. Hashing points into grid cells of side r means each point is compared only with points in its own and neighbouring cells.

OPTIMAL SOLUTION · Grid hashing with cells of side r, then Union-Find

TIME	SPACE
O(n) expected for evenly spread points	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Report the size of the largest cluster.

Count points per root after the unions (a Counter of find(i)). The unions are the same.

FOLLOW-UP 2 Points arrive one at a time.

Insert the point into its grid cell, union it with every point within r in the 9 surrounding cells, and keep the component count: components += 1 - (number of distinct roots merged).

FOLLOW-UP 3 Millions of points in 3D or higher.

The grid generalises (3^d neighbouring cells), but for high dimensions use a k-d tree or locality-sensitive hashing; or DBSCAN, which is this problem with a minimum neighbour count. **Cost:** grid hashing visits 3^d neighbouring cells per point, O(n * 3^d); a k-d tree builds in O(n log n) with O(n^(1 - 1/d) + k) range queries.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Clusters of 2D Points Within a
# Radius
#
# This question tests graph traversal skills, specifically
# identifying connected components among a set of 2D
# points using distance-based adjacency. It assesses the
# ability to model a geometric problem as a graph and
# apply DFS, BFS, or Union-Find, a common evaluation of
# algorithmic thinking in coding interviews. It also
# probes awareness of efficiency trade-offs when scaling
# to large point sets.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Report the size of the largest cluster.
# 2. Points arrive one at a time.
# 3. Millions of points in 3D or higher.
#
# INTERVIEW NOTES
# Model it as a graph where points within r are neighbours
# and count connected components with Union-Find. The all-
# pairs check is O(n^2); to speed it up, hash points into
# square cells of side r so that only points in the same
# or an adjacent cell can be within r. Compare squared
# distances to r * r to avoid square roots.
#
# BRUTE FORCE (say it first): Union-Find over all pairs of
# points
# Compare every pair, union the ones within r, and count
# the roots. Correct and simple; the quadratic pair loop
# is the cost. Time O(n^2). Space O(n).


from collections import Counter, defaultdict


# BRUTE FORCE (say it first): Union-Find over all pairs of
# points | time O(n^2)
def _find_factory(n):
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    return parent, find


# HELPERS for the base question
# FOLLOW-UP 2: Points arrive one at a time.
# Answer: Insert the point into its grid cell, union it
# with every point within r in the 9 surrounding cells,
# and keep the component count: components += 1 - (number
# of distinct roots merged).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class Clusters:
    """Points arrive one by one; count the clusters."""

    def __init__(self, r):
        self.r = r
        # grid cell (side r) -> indices of points inside
        self.cells = defaultdict(list)
        self.points, self.parent = [], []
        self.components = 0

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def near(self, p, q):
        dx, dy = p[0] - q[0], p[1] - q[1]
        return dx * dx + dy * dy <= self.r * self.r

    def add(self, x, y):
        i = len(self.points)
        self.points.append((x, y))
        self.parent.append(i)
        self.components += 1
        size = self.r or 1
        cx, cy = x // size, y // size
        self.cells[(cx, cy)].append(i)
        # look only at the 9 cells around the new point
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                key = (cx + dx, cy + dy)
                cell = self.cells.get(key, ())
                for j in cell:
                    q = self.points[j]
                    if j != i and self.near((x, y), q):
                        a, b = self.find(i), self.find(j)
                        if a != b:
                            self.parent[a] = b
                            self.components -= 1
        return self.components

    def largest(self):
        sizes = Counter(self.find(i)
                        for i in range(len(self.points)))
        return max(sizes.values(), default=0)


def clusters_all_pairs(points, r):
    parent, find = _find_factory(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if (points[i][0] - points[j][0]) ** 2 + (
                points[i][1] - points[j][1]
```

```
            ) ** 2 <= r * r:
                parent[find(i)] = find(j)
    return len({find(i) for i in range(len(points))})


# BASE QUESTION (optimal): Grid hashing with cells of side
# r, then Union-Find
# Idea: Put each point in the cell (x // r, y // r). A
# point within distance r of another can only be in the
# same cell or one of the 8 neighbouring cells, so compare
# only with those.
def count_clusters(points, r):
    c = Clusters(r)
    for x, y in points:
        c.add(x, y)
    return c.components


# FOLLOW-UP 1: Report the size of the largest cluster.
# Answer: Count points per root after the unions (a
# Counter of find(i)). The unions are the same.
def largest_cluster(points, r):
    c = Clusters(r)
    for x, y in points:
```

```
        c.add(x, y)
    return c.largest()


if __name__ == "__main__":
    line = [(0, 0), (3, 0), (6, 0), (20, 20)]
    assert count_clusters(line, 3) == 2
    assert largest_cluster(line, 3) == 3
    assert count_clusters([], 3) == 0
    # online: the count changes as points arrive
    c = Clusters(2)
    assert [c.add(*p) for p in [(0, 0), (10, 10), (1, 1),
                                (5, 5)]] == [1, 2, 2, 3]
    # (3, 3) is alone; (4, 4) then bridges it to (5, 5)
    assert c.add(3, 3) == 4
    assert c.add(4, 4) == 3
    assert count_clusters([(1, 1), (1, 1)], 0) == 1
```

FUNCTION AND TEST CASES

Signature

```
def countClusters(points: list[list[int]], r: int) -> int:
```

Sample call

```
countClusters([[0, 0], [1, 0], [10, 10]], 2)
```

Test cases

Input	Expected
([[0, 0], [1, 0], [10, 10]], 2)	2
([[0, 0], [3, 0], [6, 0]], 3)	1

ALSO REPORTED · 2 more

Count Connected Clusters of Two-Dimensional Points | PracHub | Technical Screen

https://prachub.com/coding-questions/count-connected-clusters-of-two-dimensional-points

2D Points Connected Components by Radius | 1Point3Acres | phone-screen | asked 2026-08-04 | Low

ALSO REPORTED · 2 more

https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

DFS, BFS, or Union-Find all fit the core connected-components model.
For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.
Clarify squared-distance comparison to avoid unnecessary square roots.

Preparation: Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

STUDY PATH: question 5 of 167 (Core 90)

Priority score 3.59 | Graph traversal: BFS and DFS | asked 633 days ago

MEDIUM | PracHub | Technical Screen | 5 of 8

★ TOP 75: rank 27 in the Medium and Hard priority list

★ Solve Grid and Counting Problems

POSTED 2025-01-25 | 633 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 64.8 | 30 DAYS 33 | PRIORITY 18/228

ABOUT THIS QUESTION

Topic: Core 90 - Graph traversal: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #200 Number of Islands

LeetCode match: LC 200 Number of Islands (+ a counting task)

Posted on PracHub: 2025-01-25 (updated 2026-04-24)

Original: https://prachub.com/coding-questions/solve-grid-and-counting-problems

QUESTION

A software engineering intern phone screen included two coding tasks: 1.

Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.

Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character. Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.

Constraints

0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length

EXAMPLES

```
Input: []
```

Expected Output:

Explanation: Empty grid contains no land, so there are no islands.

```
Input: [['1']]
```

QUESTION

Expected Output:

Explanation: A single land cell forms one island.

Hints

Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.

Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.

Constraints

0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type

Examples

Input: []

Expected Output:

Explanation: No letters means it is impossible to form GOOGLE.

Input: ['G','O','O','G','L','E']

Expected Output:

Explanation: There is exactly one way to choose all six letters.

Hints

First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1 L and 1 E.

APPROACH

Islands: DFS/BFS or union-find over '1' cells. Second counting task: use prefix-sum + hash map or math counting per the statement.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two tasks. Islands: four-directional adjacency, grid up to 300 x 300, empty grid allowed. Counting: choose letters from an array so the chosen ones can be rearranged to spell GOOGLE, each index used at most once, different indices are different choices.

BRUTE FORCE TO SAY OUT LOUD

Islands: none needed, DFS is the standard. Counting: enumerate all 6-element subsets of the array and test each one. O(n^6).

OPTIMAL IDEA

Islands: scan cells, start a DFS or BFS (iterative, to avoid recursion limits) from each unvisited land cell, count the starts. O(mn). Counting: GOOGLE needs G twice, O twice, L once, E once. The number of ways is C(g, 2) * C(o, 2) * l * e, where g, o, l, e are the counts of those letters in the array, since the chosen multiset is fixed and only the indices vary. O(n) to count.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Islands: empty grid, a single land cell, diagonal-only neighbors (separate islands). Counting: missing a letter (0), exactly one G (0), lots of copies, an empty array.

LIKELY FOLLOW-UPS

Islands: add land one cell at a time with Union-Find. Counting: a different target word, or count ways when letters have to appear in order (subsequence DP).

PITFALLS

Treating diagonal cells as connected. Counting arrangements instead of choices of indices.

HOW TO THINK ABOUT IT

Islands: every unvisited land cell starts a new island; mark everything connected to it. Counting: the target multiset is fixed (G twice, O twice, L once, E once), so the number of ways to choose the indices is a product of binomial coefficients, one factor per distinct letter. Do not enumerate arrangements.

Basis: Two tasks: LeetCode 200 (Number of Islands) and counting the ways to choose letters from an array that can be rearranged into GOOGLE (each index used at most once).

BRUTE FORCE · Islands: recursive flood fill

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == "0" or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
```

TIME	SPACE
O(m * n)	O(m * n) recursion

WHY IT IS SLOW

Time is O(m * n), but the recursion can be m * n deep on a snake-shaped island and overflow the stack. A queue or an explicit stack avoids that.

BRUTE FORCE · Counting: enumerate every 6-element selection

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    count = 0
    for idx in combinations(range(len(a)), 6):
        letters = sorted(a[i] for i in idx)
        if letters == sorted("GGOOLE"):
            count += 1
    return count
```

BRUTE FORCE · Counting: enumerate every 6-element selection

TIME	SPACE
O(C(n, 6) * 6)	O(1)

WHY IT IS SLOW

C(n, 6) selections. The letters needed are fixed, so the answer is C(#G, 2) * C(#O, 2) * #L * #E from one pass of counts.

OPTIMAL SOLUTION · Islands: iterative BFS

TIME	SPACE
O(m * n)	O(min(m, n)) queue

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Counting: product of binomial coefficients

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Any target word, not only GOOGLE.

Count the needed letters of the word; the answer is the product over distinct letters of C(available, needed).

FOLLOW-UP 2 The letters must appear in order (a subsequence of the array spells the word).

Then it is the 'distinct subsequences' DP: ways[j] counts the ways to spell the first j letters; scan the array and update j from high to low. O(n * len(word)).

FOLLOW-UP 3 Islands arrive one cell at a time.

Use Union-Find: each new land cell makes the count go up by one and each union with a neighbouring different island makes it go down by one (LeetCode 305).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve Grid and Counting Problems
#
# A software engineering intern phone screen included two
# coding tasks: 1. **Count islands in a grid** Given an m
# x n grid of characters where '1' represents land and '0'
# represents water, return the number of islands. An
```

```
# island is formed by connecting adjacent land cells
# horizontally or vertically. You may assume all four
# edges of the grid are surrounded by water. 2. **Count
# ways to form a target word** Given an array of uppercase
# letters, count how many different ways you can choose
# letters from the array so that the chosen letters can be
# rearranged to spell GOOGLE. Each array element can be
# used at most once, and letters at different indices are
# considered different choices even if they contain the
# same character. Design efficient algorithms for both
# tasks, and discuss the time and space complexity of your
# approach.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Any target word, not only GOOGLE.
# 2. The letters must appear in order (a subsequence of
# the array spells the word).
# 3. Islands arrive one cell at a time.
#
# INTERVIEW NOTES
# Islands: every unvisited land cell starts a new island;
# mark everything connected to it. Counting: the target
# multiset is fixed (G twice, O twice, L once, E once), so
# the number of ways to choose the indices is a product of
# binomial coefficients, one factor per distinct letter.
# Do not enumerate arrangements.
#
# BRUTE FORCE (say it first): Islands: recursive flood
# fill
# Scan the grid; at each unvisited land cell, count one
# island and recurse into the four neighbours. Correct and
# the textbook answer; deep recursion can overflow on a
# large snake-like island. Time O(m * n). Space O(m * n)
# recursion.


from collections import Counter, deque
from math import comb
```

```
# BRUTE FORCE (say it first): Islands: recursive flood
# fill | time O(m * n)
def islands_recursive(grid):
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen = set()

    def fill(r, c):
        if (
            not (0 <= r < m and 0 <= c < n)
            or grid[r][c] != "1"
            or (r, c) in seen
        ):
            return
        seen.add((r, c))
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            fill(r + dr, c + dc)

    count = 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == "1" and (r, c) not in seen:
                count += 1
                fill(r, c)
    return count
```

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): Islands: iterative BFS
# Idea: Same scan, but spread with a queue instead of
# recursion, so the depth of an island cannot overflow the
# stack. Marking visited cells in place (or with a set)
# keeps each cell processed once.
def num_islands(grid):
    # '1' is land: a BFS marks each island once
```

```
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            queue = deque([(r, c)])
            while queue:
                x, y = queue.popleft()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < m and 0 <= b < n):
                        continue
                    land = grid[a][b] == "1"
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        queue.append((a, b))
    return count


# PART 2: count GOOGLE in the array
# FOLLOW-UP 1: Any target word, not only GOOGLE.
# Answer: Count the needed letters of the word; the answer
# is the product over distinct letters of C(available,
# needed).
def ways_to_spell(letters, word="GOOGLE"):
    # choose which array positions to use: for every
    # needed
    # letter choose that many of the available copies
    have, need = Counter(letters), Counter(word)
    total = 1
    for ch, k in need.items():
        total *= comb(have[ch], k)
    return total


# FOLLOW-UP 2: The letters must appear in order (a
# subsequence of the array spells the word).
# Answer: Then it is the 'distinct subsequences' DP:
# ways[j] counts the ways to spell the first j letters;
# scan the array and update j from high to low. O(n *
```

```
# len(word)).
def ways_in_order(letters, word="GOOGLE"):
    # the letters must appear in order (a subsequence):
    # the
    # "distinct subsequences" DP, updated from high to low
    ways = [1] + [0] * len(word)
    for ch in letters:
        for j in range(len(word), 0, -1):
            if word[j - 1] == ch:
                ways[j] += ways[j - 1]
    return ways[len(word)]


if __name__ == "__main__":
    assert num_islands([]) == 0
```

```
assert num_islands([["1"]]) == 1
grid = ["11000", "11000", "00100", "00011"]
assert num_islands(grid) == 3
assert ways_to_spell([]) == 0
assert ways_to_spell(list("GOOGLE")) == 1
# three O's: choose 2 of them -> 3 ways
assert ways_to_spell(list("GOOOGLE")) == 3
# an extra G and an extra E: 3 * 2 = 6 ways
assert ways_to_spell(list("GGGOOLEE")) == 3 * 2
assert ways_to_spell(list("GOGLE")) == 0
# any target word
assert ways_to_spell(list("AABB"), "AB") == 4
assert ways_in_order(list("GOOGLE")) == 1
assert ways_in_order(list("GGOOGLE")) == 2
assert ways_in_order(list("GOGOLE")) == 0
assert ways_in_order(list("EGOOGL")) == 0
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[str]]) -> int:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`0`
`[['1']]`	`1`

STUDY PATH: question 6 of 167 (Core 90)

Priority score 3.17 | Graph traversal: BFS and DFS | asked 250 days ago

MEDIUM | IN APP LC 127 | PracHub | Technical Screen | 6 of 8

Find shortest relationship path using BFS

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 41.8 | 30 DAYS 0 | PRIORITY 87/228

ABOUT THIS QUESTION

Topic: Core 90 - Graph traversal: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #127 Word Ladder

LeetCode match: LC 127 Word Ladder (BFS shortest path) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/find-shortest-relationship-path-using-bfs

QUESTION

Overview: This question evaluates proficiency in graph representations and traversal algorithms—specifically BFS and adjacency-structure construction—for computing unweighted shortest paths and reasoning about reachability.

APPROACH

Build an adjacency list, BFS from the source with a parent map, then reconstruct the path; O(V+E). Consider bidirectional BFS.

HOW TO THINK ABOUT IT

Unweighted means every hop costs the same, so breadth-first search finds the fewest hops: it explores all people one hop away, then two hops, and the first time it reaches the target is the shortest path. Keep a parent map to rebuild the path and a visited set so nobody is queued twice.

Basis: LeetCode 127 (Word Ladder) idea: shortest path in an unweighted graph of people. Return the list of people from source to target, or None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = None
def dfs(node, path):
    global best
    if node == target:
        if not best or len(path) < len(best):
            best = list(path)
        return
    for nxt in graph[node]:
        if nxt not in path:
            dfs(nxt, path + [nxt])
dfs(source, [source])
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

The number of simple paths is exponential. Breadth-first search reaches the target along a shortest path first, O(V + E), and bidirectional BFS explores far fewer nodes on big graphs.

OPTIMAL SOLUTION · Breadth-first search with a parent map

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is huge (a social network).

Bidirectional BFS searches from both ends and stops when the frontiers meet; with branching factor b and distance d it explores about 2 * b^(d/2) nodes instead of b^d.

FOLLOW-UP 2 Return every shortest path, or count them.

Keep, for each node, its distance and a list of all parents at that distance; counting is a DP over the BFS layers, and listing the paths is a DFS over the parent lists.

FOLLOW-UP 3 Relationships have different strengths (weights).

BFS is no longer right: use Dijkstra with a heap for non-negative weights. Keep the same parent map to rebuild the path.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find shortest relationship path using BFS
#
# This question evaluates proficiency in graph
# representations and traversal algorithms—specifically
# BFS and adjacency-structure construction—for computing
# unweighted shortest paths and reasoning about
# reachability.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is huge (a social network).
# 2. Return every shortest path, or count them.
# 3. Relationships have different strengths (weights).
#
# INTERVIEW NOTES
# Unweighted means every hop costs the same, so breadth-
# first search finds the fewest hops: it explores all
# people one hop away, then two hops, and the first time
```

```
# it reaches the target is the shortest path. Keep a
# parent map to rebuild the path and a visited set so
# nobody is queued twice.
#
# BRUTE FORCE (say it first): Depth-first search over all
# simple paths
# Try every simple path from the source and keep the
# shortest that reaches the target. Correct and the
# oracle; exponential in the size of the graph. Time
# exponential. Space O(path).


import heapq
from collections import defaultdict, deque


# BRUTE FORCE (say it first): Depth-first search over all
# simple paths | time exponential
def shortest_relationship_dfs(adj, s, t):
    best = [None]

    def go(u, path):
        if best[0] is not None and len(path) >= len(
            best[0]
        ):
            return
        if u == t:
            best[0] = list(path)
            return
        for v in adj.get(u, ()):
            if v not in path:
                path.append(v)
                go(v, path)
                path.pop()

    go(s, [s])
    return best[0]


# BASE QUESTION (optimal): Breadth-first search with a
# parent map
# Idea: Queue the source and mark it visited. Pop a
# person, queue each unvisited neighbour with parent =
# this person, and stop when the target is queued. Follow
# parents back to the source and reverse to get the path.
def shortest_path(adj, s, t):
    # BFS: the first time we reach a node is by a shortest
    # path
    if s == t:
        return [s]
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in parent:
                parent[v] = u
                if v == t:
                    # walk the parent links back to s
                    path = [v]
                    while parent[path[-1]] is not None:
                        path.append(parent[path[-1]])
                    return path[::-1]
                queue.append(v)
    return None


# FOLLOW-UP 1: The graph is huge (a social network).
# Answer: Bidirectional BFS searches from both ends and
# stops when the frontiers meet; with branching factor b
# and distance d it explores about 2 * b^(d/2) nodes
# instead of b^d.
def bidirectional(adj, s, t):
    # a huge graph: search from both ends, expand the
    # smaller
    # side, stop when the searches meet
    if s == t:
        return 0
    dist_s, dist_t = {s: 0}, {t: 0}
    front_s, front_t = [s], [t]
    while front_s and front_t:
        if len(front_s) > len(front_t):
            front_s, front_t = front_t, front_s
            dist_s, dist_t = dist_t, dist_s
        nxt = []
        for u in front_s:
            for v in adj.get(u, ()):
                if v in dist_t:
                    return dist_s[u] + 1 + dist_t[v]
                if v not in dist_s:
                    dist_s[v] = dist_s[u] + 1
                    nxt.append(v)
        front_s = nxt
    return None


# FOLLOW-UP 2: Return every shortest path, or count them.
# Answer: Keep, for each node, its distance and a list of
# all parents at that distance; counting is a DP over the
# BFS layers, and listing the paths is a DFS over the
# parent lists.
def all_shortest_paths(adj, s, t):
    # every shortest path: remember ALL parents at the
    # best level
    dist, queue = {s: 0}, deque([s])
    parents = defaultdict(list)
    while queue:
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
            if dist[v] == dist[u] + 1:
                parents[v].append(u)
    if t not in dist:
        return []
    paths = []

    def build(node, tail):
        if node == s:
            paths.append([s] + tail)
            return
        for p in parents[node]:
            build(p, [node] + tail)

    build(t, [])
    return paths


# FOLLOW-UP 3: Relationships have different strengths
# (weights).
# Answer: BFS is no longer right: use Dijkstra with a heap
# for non-negative weights. Keep the same parent map to
# rebuild the path.
def weighted_path(graph, s, t):
    # strengths differ: Dijkstra, same parent map
    dist, parent = {s: 0}, {s: None}
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, ()):
            if d + w < dist.get(v, float("inf")):
```

```
                dist[v], parent[v] = d + w, u
                heapq.heappush(heap, (d + w, v))
    if t not in dist:
        return None
    path = [t]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return dist[t], path[::-1]


if __name__ == "__main__":
    adj = {"a": ["b", "c"], "b": ["d"], "c": ["d"],
           "d": ["e"], "e": []}
    assert shortest_path(adj, "a", "e") in (
        ["a", "b", "d", "e"], ["a", "c", "d", "e"])
    assert shortest_path(adj, "a", "a") == ["a"]
    assert shortest_path(adj, "e", "a") is None
    both_ways = {"a": ["b", "c"], "b": ["a", "d"],
                 "c": ["a", "d"], "d": ["b", "c", "e"],
                 "e": ["d"]}
```

```
    assert bidirectional(both_ways, "a", "e") == 3
    assert bidirectional(both_ways, "a", "a") == 0
    paths = all_shortest_paths(adj, "a", "e")
    assert sorted(paths) == [["a", "b", "d", "e"],
                             ["a", "c", "d", "e"]]
    graph = {"a": [("b", 1), ("c", 5)], "b": [("c", 1)]}
    cheapest = weighted_path(graph, "a", "c")
    assert cheapest == (2, ["a", "b", "c"])
    assert weighted_path(graph, "c", "a") is None
```

FUNCTION AND TEST CASES

Signature

```
def solution(relationships: list[list[int]], start: int, t
arget: int) -> int:
```

Sample call

```
solution([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')
```

Test cases

Input	Expected
`([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')`	`3`
`([[1, 2], [2, 5], [1, 3], [3, 4], [4, 5]], 1, 5)`	`2`

ALSO REPORTED · 1 more

Degrees of Friendship | Exponent

https://www.aced.io/questions/3628/degrees-of-friendship

STUDY PATH: question 7 of 167 (Core 90)

Priority score 3.08 | Graph traversal: BFS and DFS | asked 445 days ago

MEDIUM | IN APP LC 207 | PracHub | Technical Screen | 7 of 8

★ TOP 75: rank 33 in the Medium and Hard priority list

★ Validate course catalog dependencies

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 30/228

ABOUT THIS QUESTION

Topic: Core 90 - Graph traversal: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #207 Course Schedule

LeetCode match: LC 207 Course Schedule

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/validate-course-catalog-dependencies

QUESTION

Design a function to validate an e-learning course catalog. You are given: (1) a set of course IDs, and (2) a list of prerequisite pairs (u, v) meaning course v requires course u. Determine if the catalog is valid: no prerequisite references a non-existent course, and there are no cycles in the dependency graph. If valid, return true and a possible ordering of courses; otherwise return false and report the set of missing course references and at least one cycle you detect. Specify your algorithm, time and space complexity.

APPROACH

Check every prerequisite refers to a known course ID, then Kahn's topological sort or DFS coloring to detect cycles.

HOW TO THINK ABOUT IT

Two separate validity rules: every course mentioned must exist, and the dependency graph must have no cycle. The first is a set lookup per pair. For the second, Kahn's algorithm removes courses with no unmet prerequisites; anything left over is on a cycle or depends on one, and a DFS restricted to the leftovers reveals an actual cycle to report.

Basis: LeetCode 207 / 210 (Course Schedule) plus a check for references to unknown courses. A pair (u, v) means course v requires u.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def validate(courses, prereqs):
    for u, v in prereqs:
        if u not in courses or v not in courses:
            return False  # unknown id
    graph = build_graph(prereqs)
    for c in courses:
        if reaches(graph, c, c):
            return False  # cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Every course can trigger a walk of the whole graph, O(V * (V + E)). Kahn's algorithm is O(V + E) and its leftovers also reveal a cycle to report.

OPTIMAL SOLUTION · Missing references, then Kahn's algorithm, then a cycle from the leftovers

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return all missing course references and one cycle.

The function above returns the sorted unknown ids when any exist and otherwise a cycle (a list of courses starting and ending at the same course), found by walking backwards along prerequisites inside the set of courses Kahn's algorithm could not remove (every such course has a leftover prerequisite, so the walk cannot dead-end).

FOLLOW-UP 2 Group courses into semesters (the earliest round each can be taken).

Run Kahn's algorithm level by level: all courses with no remaining prerequisites form round 1, remove them, the next set forms round 2, and so on. The number of rounds is the longest dependency chain.

FOLLOW-UP 3 Find all cycles, not just one.

Compute strongly connected components (Tarjan or Kosaraju); every component with more than one course (or a self-loop) is a cycle group to report.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Validate course catalog dependencies
#
# Design a function to validate an e-learning course
# catalog. You are given: ( 1) a set of course IDs, and (
# 2) a list of prerequisite pairs (u, v) meaning course v
# requires course u. Determine if the catalog is valid: no
# prerequisite references a non-existent course, and there
# are no cycles in the dependency graph. If valid, return
# true and a possible ordering of courses; otherwise
# return false and report the set of missing course
# references and at least one cycle you detect. Specify
# your algorithm, time and space complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return all missing course references and one cycle.
# 2. Group courses into semesters (the earliest round each
# can be taken).
# 3. Find all cycles, not just one.
#
# INTERVIEW NOTES
# Two separate validity rules: every course mentioned must
# exist, and the dependency graph must have no cycle. The
# first is a set lookup per pair. For the second, Kahn's
# algorithm removes courses with no unmet prerequisites;
# anything left over is on a cycle or depends on one, and
# a DFS restricted to the leftovers reveals an actual
# cycle to report.
#
# BRUTE FORCE (say it first): DFS from every course
# looking for a cycle
# First collect references to unknown courses; then, for
# each course, search for a path back to itself. Correct
# and the oracle; the same parts of the graph are searched
# repeatedly. Time O(V * (V + E)). Space O(V).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): DFS from every course
# looking for a cycle | time O(V * (V + E))
def catalog_valid_brute(courses, prereqs):
    cs = set(courses)
    missing = sorted(
        {
            x
            for u, v in prereqs
            for x in (u, v)
            if x not in cs
        }
    )
    if missing:
        return False
    adj = defaultdict(list)
    for u, v in prereqs:
        adj[u].append(v)

    def reaches_self(start):
        seen, stack = set(), list(adj[start])
        while stack:
            x = stack.pop()
            if x == start:
                return True
            if x not in seen:
                seen.add(x)
                stack.extend(adj[x])
        return False

    return not any(reaches_self(c) for c in cs)


# HELPERS for the base question
# FOLLOW-UP 3: Find all cycles, not just one.
# Answer: Compute strongly connected components (Tarjan or
# Kosaraju); every component with more than one course (or
# a self-loop) is a cycle group to report.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def find_cycle(known, prereqs, indeg):
    # every leftover course still has a leftover
    # prerequisite,
    # so walking backwards must eventually repeat a course
    left = {c for c in known if indeg[c] > 0}
    before = defaultdict(list)
    for u, v in prereqs:
        if u in left and v in left:
            before[v].append(u)
    x = min(left, key=str)
    walk, pos = [], {}
    while x not in pos:
        pos[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    cycle = walk[pos[x]:] + [x]
    # walked against the edges: reverse to list
    # prerequisites first
    return cycle[::-1]


# BASE QUESTION (optimal): Missing references, then Kahn's
# algorithm, then a cycle from the leftovers
# Idea: Report unknown courses (any referenced id not in
# the set). If none, run Kahn's algorithm: repeatedly
# remove courses with no remaining prerequisites, building
# an order.
# FOLLOW-UP 1: Return all missing course references and
# one cycle.
# Answer: The function above returns the sorted unknown
# ids when any exist and otherwise a cycle (a list of
# courses starting and ending at the same course), found
# by walking backwards along prerequisites inside the set
# of courses Kahn's algorithm could not remove (every such
# course has a leftover prerequisite, so the walk cannot
# dead-end).
def validate_catalog(courses, prereqs):
    # prereqs (u, v): course v requires course u.
    # Returns (ok, missing ids, order or one cycle)
    known = set(courses)
    missing = sorted({x for pair in prereqs for x in pair
                      if x not in known}, key=str)
    if missing:
        return False, missing, None
    adj, indeg = defaultdict(list), {c: 0 for c in known}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    ready = [c for c in known if indeg[c] == 0]
    queue = deque(sorted(ready, key=str))
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
```

```
                queue.append(nxt)
    if len(order) == len(known):
        return True, [], order
    return False, [], find_cycle(known, prereqs, indeg)


# FOLLOW-UP 2: Group courses into semesters (the earliest
# round each can be taken).
# Answer: Run Kahn's algorithm level by level: all courses
# with no remaining prerequisites form round 1, remove
# them, the next set forms round 2, and so on. The number
# of rounds is the longest dependency chain.
def semesters(courses, prereqs):
    # earliest round each course can be taken: Kahn by
    # levels
    adj = defaultdict(list)
    indeg = {c: 0 for c in courses}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    level = sorted(c for c in courses if indeg[c] == 0)
    rounds = []
    while level:
        rounds.append(level)
        nxt = []
        for c in level:
            for v in adj[c]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    nxt.append(v)
        level = sorted(nxt)
    return rounds


if __name__ == "__main__":
    courses = ["a", "b", "c", "d"]
    pre = [("a", "b"), ("b", "c"), ("a", "d")]
    ok, missing, order = validate_catalog(courses, pre)
    assert ok and order[0] == "a"
    assert order.index("b") < order.index("c")
    # an unknown course id
    bad = validate_catalog(courses, pre + [("zzz", "a")])
    assert bad == (False, ["zzz"], None)
    # a cycle b -> c -> b
    loop = pre + [("c", "b")]
```

```
    ok, _, cycle = validate_catalog(courses, loop)
    assert not ok and cycle[0] == cycle[-1]
    assert set(cycle) == {"b", "c"}
    rounds = semesters(courses, pre)
    assert rounds == [["a"], ["b", "d"], ["c"]]
    assert validate_catalog([], []) == (True, [], [])
```

FUNCTION AND TEST CASES

Signature

```
def solution(course_ids: list[str], prereqs: list[tuple[st
r, str]]) -> tuple[bool, list[str], list[str], list[str]]:
```

Sample call

```
solution(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])
```

Test cases

Input	Expected
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])`	`(True, ['A', 'B', 'C'], [], [])`
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C'), ('C', 'A')])`	`(False, [], [], ['A', 'B', 'C', 'A'])`

STUDY PATH: question 8 of 167 (Core 90)

Priority score 3.55 | Graph traversal: BFS and DFS | asked 128 days ago

HARD | PracHub | Technical Screen | 8 of 8

Infection Spread on a Grid (Cellular Automaton)

POSTED 2026-06-14 | 128 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 0** | **PRIORITY 65/228**

ABOUT THIS QUESTION

Topic: Core 90 - Graph traversal: BFS and DFS

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #994 Rotting Oranges, LeetCode #289 Game of Life

LeetCode match: LC 994 Rotting Oranges / LC 289 Game of Life (inferred from overview)

Posted on PracHub: 2026-06-14 (updated 2026-06-24)

Original: https://prachub.com/coding-questions/infection-spread-on-a-grid-cellular-automaton

QUESTION

Overview: This question tests a candidate's ability to implement and optimize a synchronous cellular automaton simulation on a 2D grid, assessing proficiency in BFS-based propagation, state machine design, and incremental computation. It falls under algorithms and grid simulation, evaluating practical coding skill and the ability to handle progressively complex constraints including recovery delays, barriers, and early-termination fixed-point detection.

APPROACH

Keep a frontier of newly changed cells and update only their neighbors each step (multi-source BFS style) instead of rescanning the grid; use a state machine for cell transitions.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy has only the overview, so ask for the rules: the states (healthy, infected, recovered or immune), how infection spreads (four neighbors per step, synchronously), the recovery delay, whether barriers block spread, and what to return (the number of steps until nothing changes, or the final grid).

BRUTE FORCE TO SAY OUT LOUD

Rescan the entire grid on every step and compute the next grid from the old one. O(steps * mn).

OPTIMAL IDEA

Keep a frontier of cells that changed in the last step. Each step only their neighbors can change, so process the frontier like a multi-source BFS and update a separate next frontier to keep the update synchronous. Store the infection time per cell to apply the recovery delay (recover when step minus infected time reaches the delay). Stop when the frontier is empty and no recoveries are pending, since the grid has reached a fixed point. O(mn) overall.

DRY-RUN THESE CASES

No infected cells (0 steps), an isolated healthy cell behind barriers, a delay of 1, all cells infected at the start, two sources reaching a cell at the same time.

LIKELY FOLLOW-UPS

Different neighborhoods, probabilistic spread, or very large grids (sparse frontier).

PITFALLS

Updating cells in place during a step, which makes the spread asynchronous. Missing pending recoveries when deciding to stop.

INTERVIEW-DAY PLAN

Keep a frontier of cells that changed in the last step. Each step only their neighbors can change, so process the frontier like a multi-source BFS and update a separate next frontier to keep the update synchronous. Store the infection time per cell to apply the recovery delay (recover when step minus infected time reaches the delay). Stop when the frontier is empty and no recoveries are pending, since the grid has reached a fixed point. O(mn) overall.

DRY-RUN THESE CASES

No infected cells (0 steps), an isolated healthy cell behind barriers, a delay of 1, all cells infected at the start, two sources reaching a cell at the same time.

LIKELY FOLLOW-UPS

Different neighborhoods, probabilistic spread, or very large grids (sparse frontier).

PITFALLS

Updating cells in place during a step, which makes the spread asynchronous. Missing pending recoveries when deciding to stop.

HOW TO THINK ABOUT IT

A synchronous cellular automaton is a BFS in disguise: the step number is the BFS level. Only cells next to cells that just changed can change next, so keep a frontier instead of rescanning the grid. Always compute the next state from the old state (not in place) so all cells update together.

Basis: Written. Assumed rules (confirm them): '.' healthy, 'I' infected, 'X' barrier; each step every healthy cell with an infected 4-neighbour becomes infected, all at once; the answer is the number of steps until nothing changes.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    steps = 0
    while True:
        nxt = copy(grid)
        for r, c in all_cells(grid):
            if grid[r][c] == HEALTHY:
                if sick_neighbour(grid, r, c):
                    nxt[r][c] = INFECTED
        if nxt == grid:
            return steps
        grid = nxt
        steps += 1
```

TIME	SPACE
O(steps * m * n)	O(m * n)

WHY IT IS SLOW

Every step scans the whole grid even when only a thin edge is changing, O(steps * m * n). Keeping a frontier of newly infected cells (BFS) touches each cell once, O(m * n).

OPTIMAL SOLUTION · Frontier BFS (only recently infected cells spread)

TIME	SPACE
O(m * n)	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add a recovery delay: an infected cell recovers after d steps and cannot spread afterwards.

A cell infects every healthy neighbor on the very next step, so recovery does not change how long spreading takes; it only changes what the grid looks like at time t. Store the step at which each cell was infected (the BFS level) and derive the state at any time: healthy, infected, or recovered when t minus its infection step is at least d.

FOLLOW-UP 2 Why can you stop early?

When the next frontier is empty no cell can change in any later step (the update only depends on the frontier), so the grid is at a fixed point. In a variant with pending recoveries keep going until those are done too.

FOLLOW-UP 3 Make the spread probabilistic.

A frontier cell may fail to infect a neighbor this step, so it stays in the frontier. Keep frontier cells until all their healthy neighbors are infected, and draw a random number per (cell, neighbor, step). The expected number of steps now needs simulation.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Infection Spread on a Grid (Cellular
# Automaton)
#
# This question tests a candidate's ability to implement
# and optimize a synchronous cellular automaton simulation
# on a 2D grid, assessing proficiency in BFS-based
# propagation, state machine design, and incremental
# computation. It falls under algorithms and grid
# simulation, evaluating practical coding skill and the
# ability to handle progressively complex constraints
# including recovery delays, barriers, and early-
# termination fixed-point detection.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add a recovery delay: an infected cell recovers after
# d steps and cannot spread afterwards.
# 2. Why can you stop early?
# 3. Make the spread probabilistic.
#
# INTERVIEW NOTES
# A synchronous cellular automaton is a BFS in disguise:
# the step number is the BFS level. Only cells next to
# cells that just changed can change next, so keep a
# frontier instead of rescanning the grid. Always compute
# the next state from the old state (not in place) so all
# cells update together.
#
# BRUTE FORCE (say it first): Rescan the whole grid every
# step
# Build the next grid from the current one by checking
# every cell's four neighbors, and repeat until a step
# changes nothing. Easy to get right, so it doubles as a
# test oracle. The cost is that every step touches the
# whole grid even when only a thin edge is changing. Time
# O(steps * m * n). Space O(m * n).


# BRUTE FORCE (say it first): Rescan the whole grid every
# step | time O(steps * m * n)
def spread_steps_brute(grid):
    g = [list(r) for r in grid]
    steps = 0
    while True:
        nxt = [row[:] for row in g]
        changed = False
        for r in range(len(g)):
            for c in range(len(g[0])):
                if g[r][c] == "." and any(
                    0 <= r + dr < len(g)
                    and 0 <= c + dc < len(g[0])
                    and g[r + dr][c + dc] == "I"
                    for dr, dc in (
                        (1, 0),
                        (-1, 0),
                        (0, 1),
                        (0, -1),
                    )
                ):
                    nxt[r][c] = "I"
                    changed = True
        if not changed:
            return steps
        g, steps = nxt, steps + 1


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# TIME each cell gets infected
def infection_times(grid):
    # step at which each cell gets infected (a BFS level);
    # 'I' = infected at time 0, '.' = healthy, '#' =
    # blocked
    rows, cols = len(grid), len(grid[0])
    when = {}
    frontier = [(r, c) for r in range(rows)
                for c in range(cols) if grid[r][c] == "I"]
    for cell in frontier:
        when[cell] = 0
    step = 0
    while frontier:
        step += 1
        nxt = []
        for r, c in frontier:
            for dr, dc in STEPS:
                a, b = r + dr, c + dc
                inside = 0 <= a < rows and 0 <= b < cols
                # every healthy neighbour is infected next
                # step
                if inside and grid[a][b] == "." \
                        and (a, b) not in when:
                    when[(a, b)] = step
```

```
                    nxt.append((a, b))
        frontier = nxt
    return when


# BASE QUESTION (optimal): Frontier BFS (only recently
# infected cells spread)
# Idea: Put every initially infected cell in the frontier.
# Each step, look only at the neighbors of frontier cells,
# infect the healthy ones, and make them the next
# frontier.
def spread_steps(grid):
    # steps until nothing changes any more
    when = infection_times(grid)
    return max(when.values(), default=0)


# FOLLOW-UP 1: Add a recovery delay: an infected cell
# recovers after d steps and cannot spread afterwards.
# Answer: A cell infects every healthy neighbor on the
# very next step, so recovery does not change how long
# spreading takes; it only changes what the grid looks
# like at time t.
def grid_at(grid, t, recovery=None):
    # the grid after t steps; with a recovery delay d an
    # infected cell turns into 'R' (it spread already, so
    # the
    # timing above does not change)
    when = infection_times(grid)
    out = []
    for r, row in enumerate(grid):
        line = ""
        for c, ch in enumerate(row):
            start = when.get((r, c))
            if start is None or start > t:
                line += ch
            elif recovery and t >= start + recovery:
                line += "R"
            else:
                line += "I"
        out.append(line)
    return out
```

```
if __name__ == "__main__":
    g = ["I..", "...", "..."]
    assert spread_steps(g) == 4
    assert grid_at(g, 1) == ["II.", "I..", "..."]
    # with recovery after 1 step the source recovers first
    rec = grid_at(g, 1, recovery=1)
    assert rec == ["RI.", "I..", "..."]
    # a wall splits the grid: the far cell is never
    # infected
    wall = ["I#.", "I#."]
    assert spread_steps(wall) == 0
    assert grid_at(wall, 9)[0] == "I#."
    assert spread_steps(["..."]) == 0
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[str]) -> list[str]:
```

Sample call

```
solution(['HHH', 'HIH', 'HHH'])
```

Test cases

Input	Expected
`(['HHH', 'HIH', 'HHH'],)`	`['HIH', 'III', 'HIH']`
`(['HHH'],)`	`['HHH']`